



Data Visualization

LABORATORIO

Capitolo 0 – Introduzione all'ambiente



Contatti

Alessandra Perniciano

alessandra.pernician@unica.it



Lab. InfoLife (porta a sinistra prima dell'aula magna di Matematica)

Ricevimento:

Durante le ore di laboratorio, su Teams, via mail

Obiettivi del laboratorio

01

Vedere all'atto **pratico**
i contenuti delle lezioni
di teoria.

02

Imparare ad utilizzare tool
di data visualization, nel
nostro caso **D3**.

03

Sapere infine trasformare
dati complessi in insight
comprensibili, utilizzabili
e **visivamente accattivanti**
per gli *utenti finali*.

Come funzionerà il laboratorio

Giovedì mattina, dalle 09:00 alle 10:40, Laboratorio T

- Lezione del giorno
- Esercizi guidati sulla lezione
- Esercizi sulla lezione con supporto del tutor

Strumenti di lavoro



Useremo **WebStorm** come IDE per lo svolgimento degli esercizi e gli esami.
Useremo la libreria **D3** per creare visualizzazioni di dati dinamiche e interattive.

Perché **non** usare Visual Studio Code?

Creare un server da zero potrebbe non essere banale. WebStorm ci viene in soccorso perché mette automaticamente a disposizione un server locale senza dover fare installazioni particolari.



Cosa vedremo in laboratorio

Introduzione

Concetti base della programmazione web:

- HTML
- SVG
- CSS
- JS

D3-base

Concetti e grafici base:

- Scale
- Linechart
- Scatterplot
- Axis
- Bar chart
- Pie chart

D3-avanzato

Grafici complessi:

- Tree
- Networks
- Choropleth Maps
- Grafici collegati



01 Introduzione

HTML, SVG e CSS



Argomenti del giorno

- **Creazione di un progetto su WebStorm**
 - Concetti base di HTML
 - Creazione di SVG semplici
 - Creazione e collegamento di un file CSS
- **Esercizi sulla lezione**
 - Su E-Learning trovate l'archivio allegato a questa lezione

Basi della programmazione web



HyperText Markup Language

Linguaggio di markup per descrivere la **struttura** e il **contenuto** di un documento, utilizzando un insieme di **tag**.



Cascading Style Sheet

Linguaggio utilizzato per definire lo **stile**. In pratica dice al browser come **mostrare** gli elementi html.

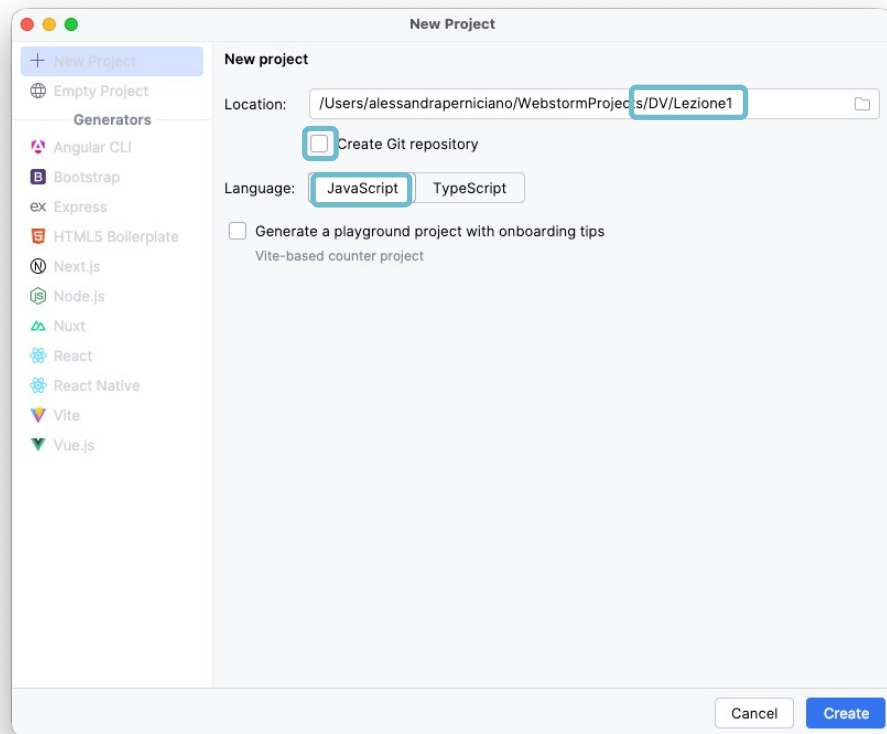


JavaScript

Linguaggio di programmazione utilizzato per rendere siti web interattivi e **dinamici**.

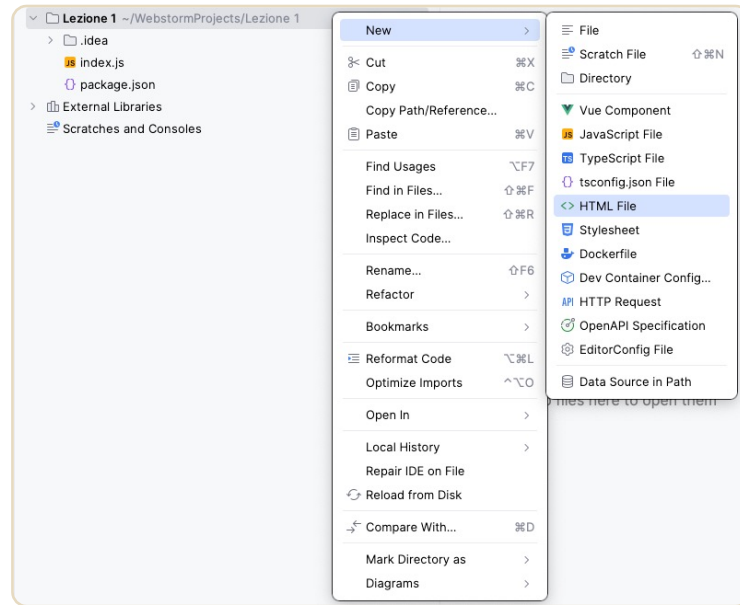
Creazione di un progetto

1. Avviamo WebStorm e creiamo un nuovo progetto.
2. Nella finestra che appare specifichiamo:
 1. **Nome del progetto** in *Location*
 2. **JavaScript** in *Language*
3. Potete togliere la spunta da *Create Git repository* se non avete intenzione di caricare le esercitazioni



Creazione di un file HTML

1. Premiamo il tasto destro sul progetto (in questo caso Lezione 1) > > New > > HTML File
2. Chiamiamo il file **index.html**



HTML

`<h1> Titolo </h1>`

Tag di apertura Tag di chiusura

Elemento

`<a href="www.google.it"> Link </a>`

Nome Valore

Attributo

Gli attributi più usati sono:

- **class:** specifica una classe (più elementi possono avere la stessa classe)
- **id:** identificatore univoco per un elemento
- **style:** stile css specificato come attributo

HTML – struttura

Ora che abbiamo creato il file, analizziamo la struttura e i tag che WebStorm ha inserito automaticamente:

- **<!DOCTYPE html>** : definisce quale versione HTML vogliamo usare. In questo caso stiamo usando HTML 5
- **<head> </head>**: il tag head contiene elementi con informazioni generali sulla pagina come dove trovare i fogli di stile, il titolo, fornire meta informazioni, etc
- **<body> </body>**: è il tag che contiene la definizione della struttura del documento

```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6  </head>
7  <body>
8
9  </body>
10 </html>
```

HTML – struttura

- Nel tag **<head>**:
 - Titolo
 - Charset
 - Autore della pagina
 - Descrizione della pagina
 - Keywords
- Nel tag **<body>**:
 - Titolo della pagina
 - Sottotitolo
 - Paragrafo
 - Svg etc...

```
<head>
  <title>Title</title>
  <meta charset="UTF-8">
  <meta name="author" content="Alessandra Perniciano">
  <meta name="description" content="Pagina per grafici">
  <link rel="stylesheet" type="text/css" href="style.css">
  <script type="text/javascript" src="index.js"></script>
</head>
```

HTML – struttura

- Nel tag **<head>**:
 - Titolo
 - Charset
 - Autore della pagina
 - Descrizione della pagina
 - Keywords
- Nel tag **<body>**:
 - Titolo della pagina
 - Sottotitolo
 - Paragrafo
 - Svg etc...

```
<body>  
  <h1> Questo è un titolo </h1>  
  <h2> Questo è un sottotitolo </h2>  
  <h3> Questo sottotitolo è ancora più piccolo</h3>  
  <h4> Più è grande il numero, meno è l'importanza e più piccolo è il font</h4>  
  <p> Questo è un paragrafo. Il corpo di un testo deve essere scritto qui e può  
    esser fatto in <i>corsivo</i> oppure in <b>grassetto</b>.  
    Posso anche <u>sottolineare</u> o fare più cose <b><i><u>insieme</u></i></b>.</p>  
  <p> se voglio posso aggiungere un link come quello di <a href="http://google.com"> Google </a></p>  
</body>  
</html>
```

HTML – tag base

Aggiungiamo del contenuto alla pagina:

- Il tag **<h1>** ci permette di scrivere dei titoli. Il numero nel tag ci indica l'importanza del titolo. Più è importante e più è in evidenza
- Con **<p>** creiamo dei paragrafi, quindi i blocchi di testo all'interno delle pagine.
- All'interno possiamo utilizzare dei tag **inline**:
 - Tag di **stile/testo**:
 - **<i>** corsivo (italic)
 - **** grassetto (bold)
 - **<u>** sottolineato (underlined).
 - Tag **funzionali**, come **<a>**, per creare dei link ipertestuali e collegarci ad altre pagine o sezioni di pagine.

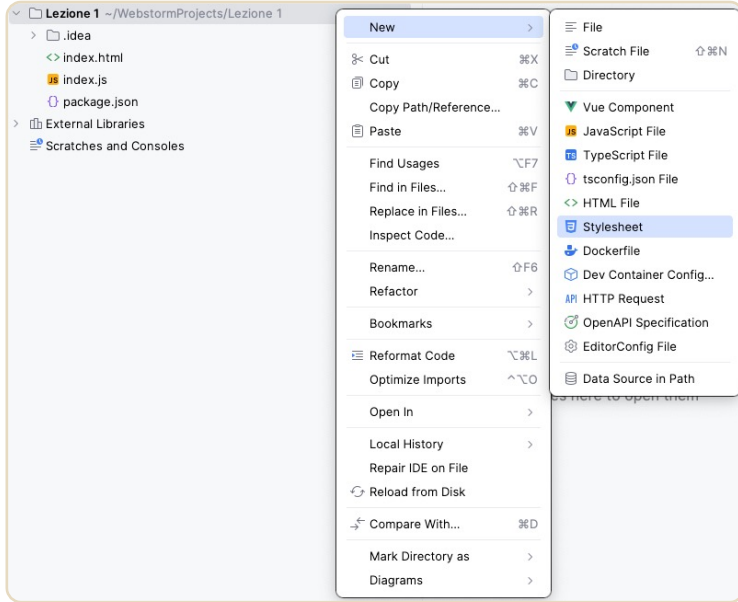
```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6  </head>
7  <body>
8    <h1>Questo è un titolo</h1>
9    <h2>E questo è un sottotitolo</h2>
10   <h3>Più il numero cresce, meno è importante il contenuto</h3>
11
12   <p>Questo è un paragrafo, e così posso scrivere in <i>corsivo</i>
13     oppure in <b>grassetto</b>. Volendo posso anche <u>sottolineare</u>.</p>
14
15   <p>In questo modo invece aggiungo un link,
16     per esempio verso <a href="http://google.com">Google</a></p>
17 </body>
18 </html>
19
20
21
22 |
```


HTML – tag base

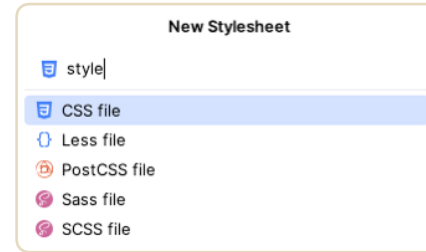
```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6  </head>
7  <body>
8    <h1>Questo è un titolo</h1>
9    <h2>E questo è un sottotitolo</h2>
10   <h3>Più il numero cresce, meno è importante il contenuto</h3>
11
12   <p>Questo è un paragrafo, e così posso scrivere in <i>corsivo</i>
13     oppure in <b>grassetto</b>. Volendo posso anche <u>sottolineare</u>.</p>
14
15   <p>In questo modo invece aggiungo un link,
16     per esempio verso <a href="http://google.com">Google</a></p>
17 </body>
18 </html>
19
20
21
22
```



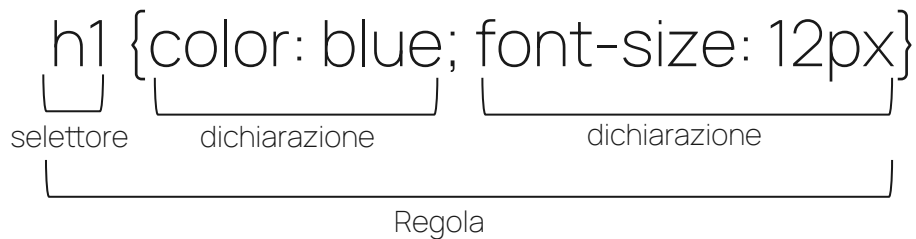
Creazione di un file CSS



1. Come per la creazione del file html, per creare un file css premiamo il tasto destro sul progetto > > New > > Stylesheet
2. Chiamiamo il file **style.css** e scegliamo come opzione CSS file



CSS - regole



Il css è formato da un insieme di regole

I selettori più comuni sono:

- Universale (*): seleziona tutti gli elementi della pagina
- Di tipo: seleziona tutti gli elementi di un dato tag HTML (es. p, h1, div)
- Di classe (.nomeClasse): seleziona gli elementi che appartengono a una classe.
- Di ID (#nomeID): seleziona un singolo elemento con uno specifico ID

CSS – selettori

<> index.html	style.css x	
1	<code>*{</code>	
2	<code>background: beige;</code>	Selettore universale
3	<code>}</code>	
4		
5	<code>h1 {</code>	
6	<code>color: blue;</code>	Selettore di tipo
7	<code>text-align: center;</code>	
8	<code>}</code>	
9		
10	<code>#singleTitle {</code>	Selettore di ID
11	<code>color: lightblue;</code>	
12	<code>}</code>	
13		
14	<code>.blue-trasparent-line{</code>	Selettore di classe
15	<code>stroke: rgba(0, 0, 255, 0.5);</code>	
16	<code>stroke-width: 2px;</code>	
17	<code>}</code>	

CSS – collegamento all'HTML

All'interno del tag <head> aggiungiamo il collegamento:

- Il tag <link> permette di collegare risorse esterne alla pagina.

```
style.css  <> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <h1>Questo è un titolo</h1>
10   <h2>E questo è un sottotitolo</h2>
11   <h3>Più il numero cresce, meno è iportante il contenuto</h3>
12
13   <p>Questo è un paragrafo, e così posso scrivere in <i>corsivo</i>
14     oppure in <b>grassetto</b>. Volendo posso anche <u>sottolineare</u>.</p>
15
16   <p>In questo modo invece aggiungo un link,
17     per esempio verso <a href="http://google.com">Google</a></p>
18 </body>
19 </html>
20
21
22
23
```

CSS – prima e dopo





Ora che abbiamo queste nozioni,
possiamo temporaneamente metterle da
parte. In questo corso infatti useremo altri
tag per creare i nostri grafici.

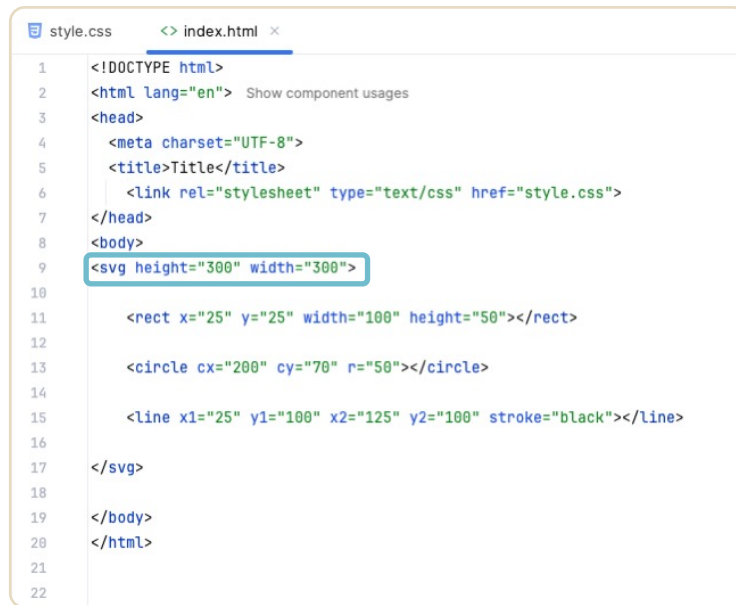


SVG – concetti base

SVG sta per **S**calable **V**ector **G**raphics, ed è un formato utilizzato per definire immagini che rimangono nitide a qualsiasi livello di zoom, a differenza delle immagini raster come PNG o JPEG.

Il tag `<svg>` è usato in HTML per creare grafica vettoriale scalabile direttamente nel browser e permette di disegnare forme geometriche, testi e immagini in modo dinamico e interattivo.

Quando si crea un elemento `<svg>` è buona norma definirne la dimensione.



```
1 <!DOCTYPE html>
2 <html lang="en"> Show component usages
3 <head>
4   <meta charset="UTF-8">
5   <title>Title</title>
6   <link rel="stylesheet" type="text/css" href="style.css">
7 </head>
8 <body>
9   <svg height="300" width="300">
10
11     <rect x="25" y="25" width="100" height="50"></rect>
12
13     <circle cx="200" cy="70" r="50"></circle>
14
15     <line x1="25" y1="100" x2="125" y2="100" stroke="black"></line>
16
17   </svg>
18
19 </body>
20 </html>
21
22
```


SVG – rettangoli

All'interno di un elemento SVG si posso disegnare tre tipi di forme geometriche base: **rettangoli**, **cerchi** e **linee**.

Il tag **<rect>** permette di creare un rettangolo.
All'interno del tag è utile definire:

- Altezza: **height**.
- Larghezza: **width**.
- (Facoltativo) Coordinate dell'origine: **x** e **y**.

Se si omettono le coordinate dell'origine, il rettangolo avrà come coordinate quelle dell'origine dell'elemento svg che lo racchiude.

```
style.css  <> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg height="300" width="300">
10
11    <rect x="25" y="25" width="100" height="50"></rect>
12
13    <circle cx="200" cy="70" r="50"></circle>
14
15    <line x1="25" y1="100" x2="125" y2="100" stroke="black"></line>
16
17  </svg>
18
19 </body>
20 </html>
21
22
```

SVG – cerchi

All'interno di un elemento SVG si posso disegnare tre tipi di forme geometriche base: **rettangoli**, **cerchi** e **linee**.

Il tag **<circle>** permette di creare un cerchio.
All'interno del tag è utile definire:

- Raggio: **r**
- (Facoltativo) Coordinate del centro: **cx** e **cy**

Se si omettono le coordinate del centro, il cerchio avrà come coordinate quelle dell'origine dell'elemento svg che lo racchiude, di fatto sarà visibile solo quarto del cerchio.

```
style.css  <> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg height="300" width="300">
10
11      <rect x="25" y="25" width="100" height="50"></rect>
12
13      <circle cx="200" cy="70" r="50"></circle>
14
15      <line x1="25" y1="100" x2="125" y2="100" stroke="black"></line>
16
17    </svg>
18
19  </body>
20 </html>
21
22
```

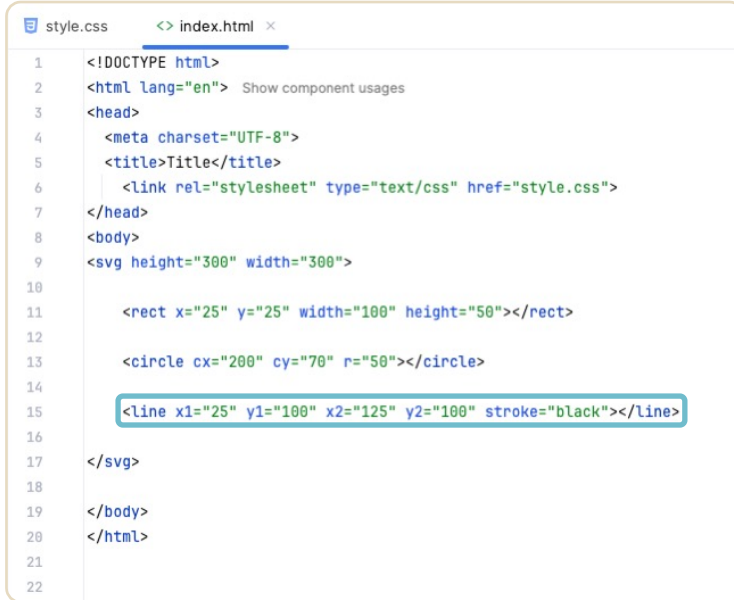
SVG – linee

All'interno di un elemento SVG si posso disegnare tre tipi di forme geometriche base: **rettangoli**, **cerchi** e **linee**.

Il tag **<line>** permette di creare una linea. All'interno del tag è utile definire:

- Coordinate del punto iniziale: **x1** e **y1**.
- Coordinate del punto finale: **x2** e **y2**.
- Colore: **stroke**.

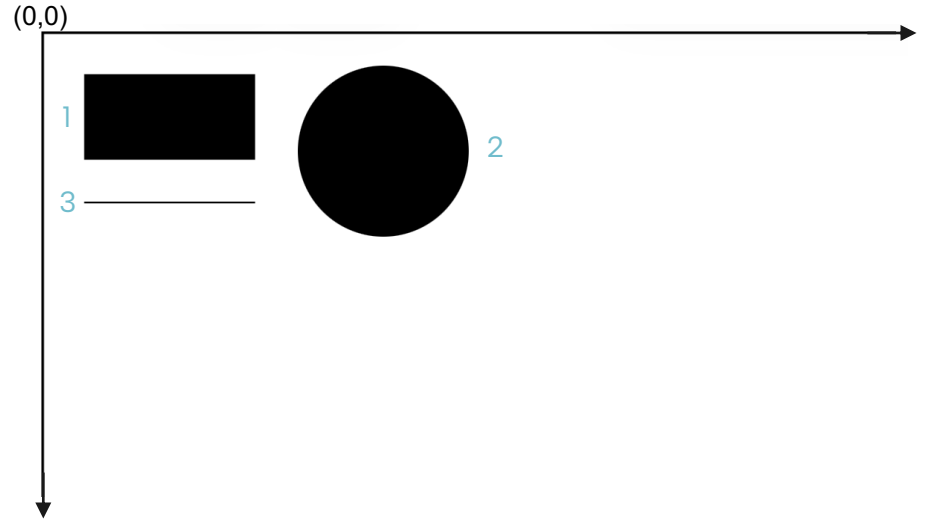
Lo stroke normalmente è il **bordo** di un elemento e può variare in colore e spessore. In una linea, possiamo vedere il contorno come la linea stessa, perciò definirne il colore definisce il colore della linea. Se si omette il punto iniziale, si inizia dall'origine (0,0)



```
1 <!DOCTYPE html>
2 <html lang="en"> Show component usages
3 <head>
4   <meta charset="UTF-8">
5   <title>Title</title>
6   <link rel="stylesheet" type="text/css" href="style.css">
7 </head>
8 <body>
9   <svg height="300" width="300">
10
11     <rect x="25" y="25" width="100" height="50"></rect>
12
13     <circle cx="200" cy="70" r="50"></circle>
14
15     <line x1="25" y1="100" x2="125" y2="100" stroke="black"></line>
16
17   </svg>
18
19 </body>
20 </html>
21
22
```

SVG – forme geometriche

```
style.css  <> index.html  x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg height="300" width="300">
10
11    1 <rect x="25" y="25" width="100" height="50"></rect>
12
13    2 <circle cx="200" cy="70" r="50"></circle>
14
15    3 <line x1="25" y1="100" x2="125" y2="100" stroke="black"></line>
16
17  </svg>
18
19 </body>
20 </html>
21
22
```



SVG – gruppi

Il tag **<g>** serve per raggruppare elementi grafici creando un **gruppo**.

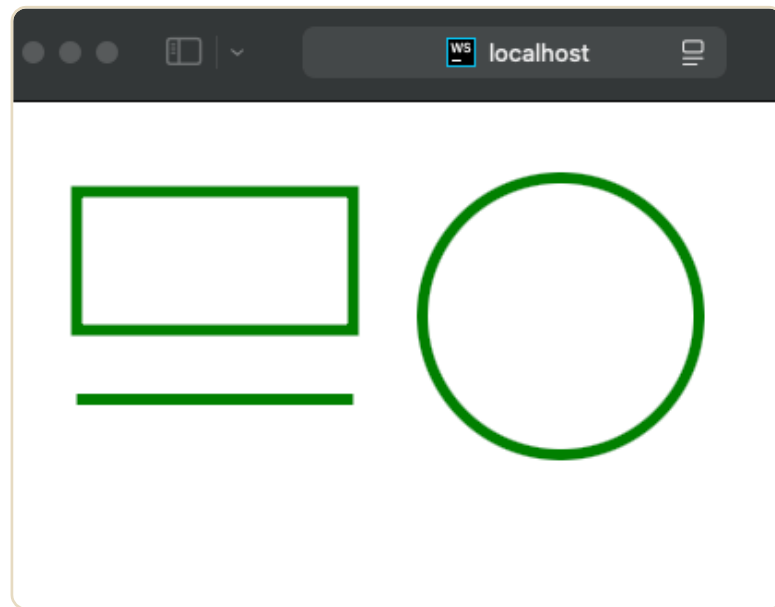
È utile per organizzare e manipolare insieme più elementi SVG, applicando per esempio **trasformazioni**, **stili** o **attributi** a tutto il gruppo invece che a ogni singolo elemento. Ci serve per gestire forme più complesse

All'interno di un tag `<svg>` possono essere presenti più gruppi.

```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg height="300" width="300">
10
11     <g fill="none" stroke="green" stroke-width="4px">
12
13       <rect x="25" y="25" width="100" height="50"></rect>
14
15       <circle cx="200" cy="70" r="50"></circle>
16
17       <line x1="25" y1="100" x2="125" y2="100" ></line>
18
19     </g>
20
21 </svg>
22
23 </body>
24 </html>
```

SVG – gruppi

```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg height="300" width="300">
10
11      <g fill="none" stroke="green" stroke-width="4px">
12
13        <rect x="25" y="25" width="100" height="50"></rect>
14
15        <circle cx="200" cy="70" r="50"></circle>
16
17        <line x1="25" y1="100" x2="125" y2="100" ></line>
18
19      </g>
20
21    </svg>
22
23  </body>
24  </html>
```



SVG – percorsi

Il tag **<path>** è uno degli elementi più potenti e flessibili per disegnare forme complesse.

Permette di creare **linee**, **curve**, **archi** e altre **geometrie avanzate** che non sono possibili con altri elementi SVG come **<rect>** o **<circle>**.

Il **<path>** disegna una forma definita da una sequenza di comandi contenuti nell'attributo **d** ("data").
Questi comandi descrivono movimenti, linee e curve.

Gli altri attributi principali sono:

- **fill**: specifica il colore di riempimento della forma.
- **stroke**: specifica il colore del bordo.
- **stroke-width**: definisce lo spessore del bordo.

```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg>
10     <g>
11       <path d="M10 150 L50 100
12         L90 120 L130 80
13         L170 140 L210 60
14         L250 100 L290 50
15         L330 120 Z"
16         stroke="blue"
17         fill="none"
18         stroke-width="2"/>
19     </g>
20 </svg>
21
22 </body>
23 </html>
```

SVG – comandi dell'attributo **d**

L'attributo **d** è composto da una serie di comandi seguiti da parametri numerici.

I comandi più comuni sono riguardano movimenti e linee:

- **M x y**: Move To – Sposta il punto di partenza del disegno (senza disegnare nulla).
Es: M10 10 sposta il punto iniziale a (10, 10).
- **L x y**: Line To – Disegna una linea dal punto corrente al punto (x, y).
Es: L100 100 traccia una linea fino al punto (100, 100).
- **H x / V y**: Horizontal/Vertical Line – Disegna una linea orizzontale o verticale.
Es: H50 traccia una linea orizzontale verso x=50.
- **Z o z**: Close Path – Chiude il percorso collegando l'ultimo punto al primo.

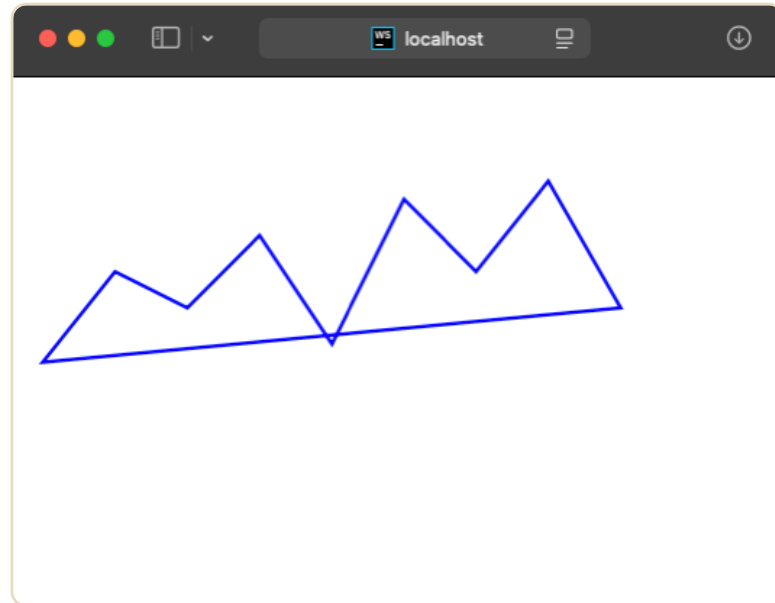
È importante sapere che l'utilizzo di lettere maiuscole o minuscole produce risultati diversi:

- **Lettere maiuscole** (es: M, L, C) usano coordinate assolute.
- **Lettere minuscole** (es: m, l, c) usano coordinate relative rispetto al punto corrente.

Nelle lezioni successive vedremo come creare diversi tipi di curve.

SVG – percorsi

```
<> index.html x
1  <!DOCTYPE html>
2  <html lang="en"> Show component usages
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg>
10     <g>
11       <path d="M10 150 L50 100
12         L90 120 L130 80
13         L170 140 L210 60
14         L250 100 L290 50
15         L330 120 Z"
16         stroke="blue"
17         fill="none"
18         stroke-width="2"/>
19     </g>
20   </svg>
21
22 </body>
23 </html>
```



SVG – combinare attributi

Noi useremo il tag **<path>** soprattutto per creare dei grafici. È possibile infatti combinarlo con altri attributi per ottenere i risultati più disparati.

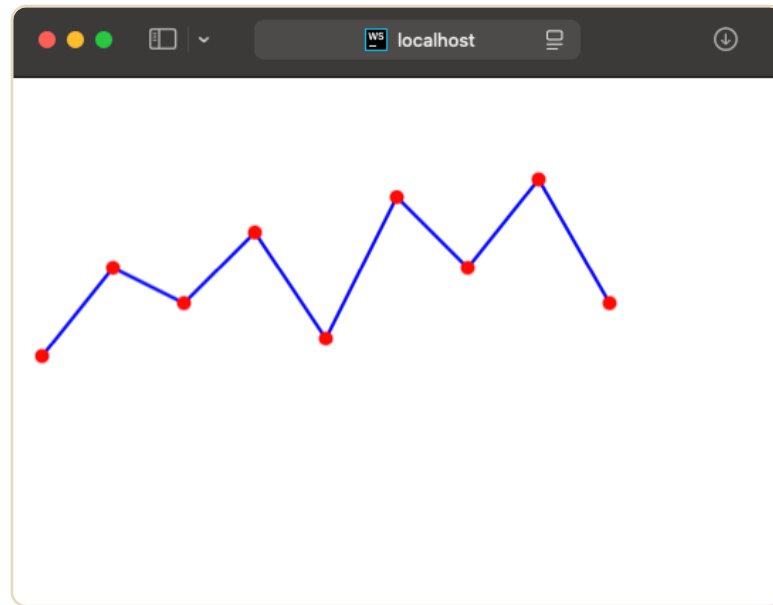
Qui vediamo un esempio con l'aggiunta dell'attributo **<circle>**.

Il risultato è una versione molto base di quello che vedremo poi essere un **linechart**.

```
<> index.html x
2  <html lang="en"> Show component usages
8  <body>
9  <svg>
10  <g>
11    <path d="M10 150 L50 100
12          L90 120 L130 80
13          L170 140 L210 60
14          L250 100 L290 50
15          L330 120"
16          stroke="blue"
17          fill="none"
18          stroke-width="2"/>
19
20    <circle cx="10" cy="150" r="4" fill="red" />
21    <circle cx="50" cy="100" r="4" fill="red" />
22    <circle cx="90" cy="120" r="4" fill="red" />
23    <circle cx="130" cy="80" r="4" fill="red" />
24    <circle cx="170" cy="140" r="4" fill="red" />
25    <circle cx="210" cy="60" r="4" fill="red" />
26    <circle cx="250" cy="100" r="4" fill="red" />
27    <circle cx="290" cy="50" r="4" fill="red" />
28    <circle cx="330" cy="120" r="4" fill="red" />
29  </g>
30 </svg>
```

SVG – combinare attributi

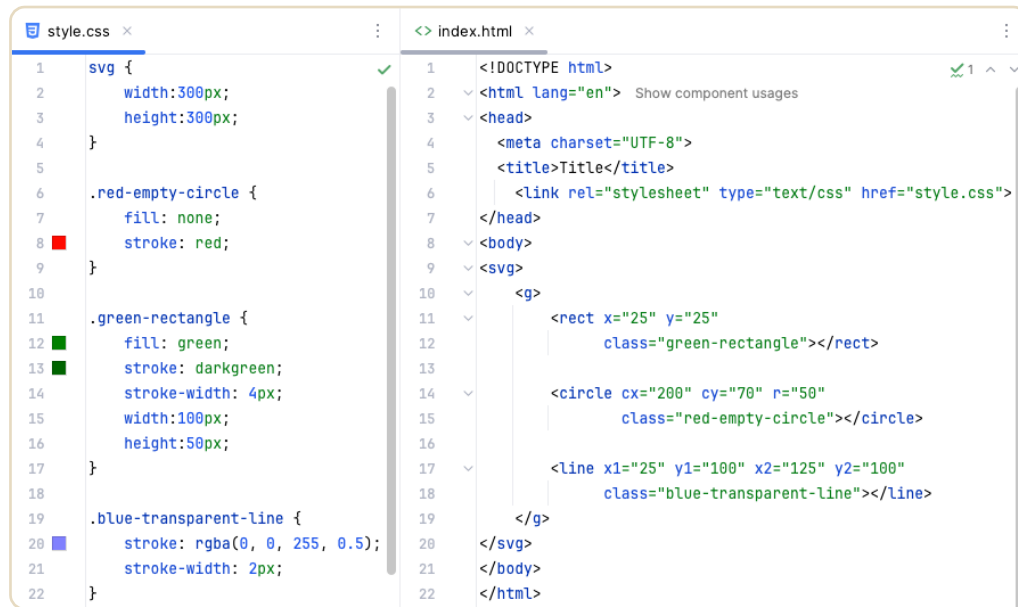
```
<> index.html x
2  <html lang="en"> Show component usages
8  <body>
9  <svg>
10 <g>
11   <path d="M10 150 L50 100
12       L90 120 L130 80
13       L170 140 L210 60
14       L250 100 L290 50
15       L330 120"
16       stroke="blue"
17       fill="none"
18       stroke-width="2"/>
19
20   <circle cx="10" cy="150" r="4" fill="red" />
21   <circle cx="50" cy="100" r="4" fill="red" />
22   <circle cx="90" cy="120" r="4" fill="red" />
23   <circle cx="130" cy="80" r="4" fill="red" />
24   <circle cx="170" cy="140" r="4" fill="red" />
25   <circle cx="210" cy="60" r="4" fill="red" />
26   <circle cx="250" cy="100" r="4" fill="red" />
27   <circle cx="290" cy="50" r="4" fill="red" />
28   <circle cx="330" cy="120" r="4" fill="red" />
29 </g>
30 </svg>
```



SVG – applicare lo stile

Per applicare dello stile tramite **CSS** a un elemento SVG si procede come per un qualunque elemento HTML.

Si possono definire regole direttamente per uno **specifico tag**, oppure definire delle **classi** da assegnare a determinati elementi del SVG.



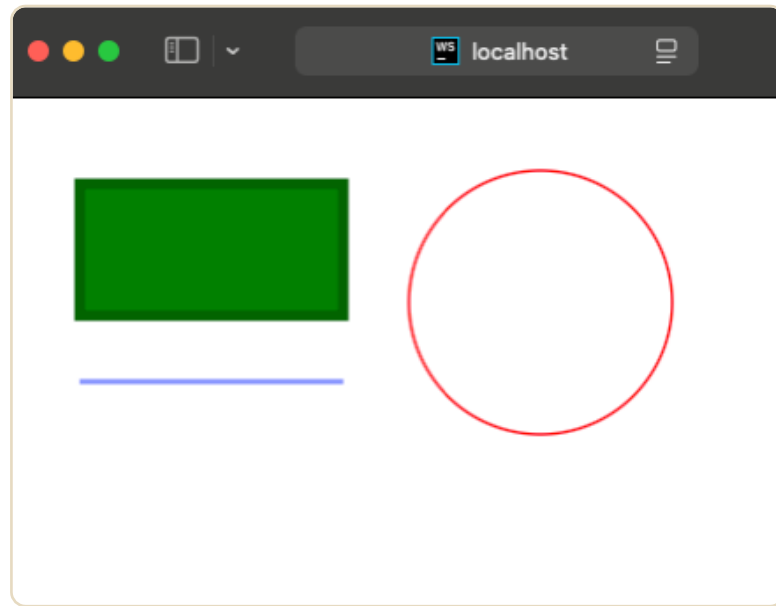
```
style.css
1  svg {
2    width:300px;
3    height:300px;
4  }
5
6  .red-empty-circle {
7    fill: none;
8    stroke: red;
9  }
10
11 .green-rectangle {
12   fill: green;
13   stroke: darkgreen;
14   stroke-width: 4px;
15   width:100px;
16   height:50px;
17 }
18
19 .blue-transparent-line {
20   stroke: rgba(0, 0, 255, 0.5);
21   stroke-width: 2px;
22 }

index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>Title</title>
6    <link rel="stylesheet" type="text/css" href="style.css">
7  </head>
8  <body>
9    <svg>
10     <g>
11       <rect x="25" y="25"
12         class="green-rectangle"></rect>
13
14       <circle cx="200" cy="70" r="50"
15         class="red-empty-circle"></circle>
16
17       <line x1="25" y1="100" x2="125" y2="100"
18         class="blue-transparent-line"></line>
19     </g>
20   </svg>
21 </body>
22 </html>
```

SVG – applicare lo stile

```
style.css x      index.html x
1  svg {
2    width:300px;
3    height:300px;
4  }
5
6  .red-empty-circle {
7    fill: none;
8    stroke: red;
9  }
10
11 .green-rectangle {
12   fill: green;
13   stroke: darkgreen;
14   stroke-width: 4px;
15   width:100px;
16   height:50px;
17 }
18
19 .blue-transparent-line {
20   stroke: rgba(0, 0, 255, 0.5);
21   stroke-width: 2px;
22 }
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
  <svg>
    <g>
      <rect x="25" y="25"
        class="green-rectangle"></rect>
      <circle cx="200" cy="70" r="50"
        class="red-empty-circle"></circle>
      <line x1="25" y1="100" x2="125" y2="100"
        class="blue-transparent-line"></line>
    </g>
  </svg>
</body>
</html>
```





Esercizi

Svolgere gli esercizi presenti nella sezione Esercitazione 1.

L'esercitazione è divisa in sottocartelle, ognuna relativa agli argomenti visti in questa lezione.

Gli esercizi sono divisi in step, preceduti da una piccola consegna. È consigliato, almeno per ora, svolgere tutti gli step, in modo da capire in quali casi il codice è corretto e in quali no.

